

RAG INGESTION & RETRIEVAL USING API

RAG Ingestion:

1. Converting Excel files (Test Cases and User Stories) to JSON for Ingestion:

The screenshot shows the RAG MongoDB API interface for the '01. Upload Excel - Test Cases' endpoint. The interface includes a sidebar with a collection list, a main panel with a table of parameters, and a response body showing a successful conversion of test cases to JSON.

Endpoint: POST `{{baseUri}}/api/upload-excel`

Parameters:

Key	Value	Description
file	testcases.xlsx	Excel file with test cases
dataType	{{dataType}}	testcases userstories
sheetName	{{sheetName}}	Sheet name inside the Excel file

Response Body (JSON):

```
{
  "success": true,
  "message": "Test cases file converted successfully",
  "outputFile": "converted-1782821605744.json",
  "output": "⚠️ Sheet \"Testcases\" not found. Using first sheet: \"testcases\"\\n✅ Converted 6354 rows from \"testcases\" into /Users/ajayuchiha/Desktop/Gen AI/Phase 2/Week 4/Day 1/rag-mongo-demo-v9/src/data/converted-1782821605744.json\\n"
}
```

The screenshot shows the RAG MongoDB API interface for the '02. Upload Excel - User Stories' endpoint. The interface includes a sidebar with a collection list, a main panel with a table of parameters, and a response body showing a successful conversion of user stories to JSON.

Endpoint: POST `{{baseUri}}/api/upload-excel`

Parameters:

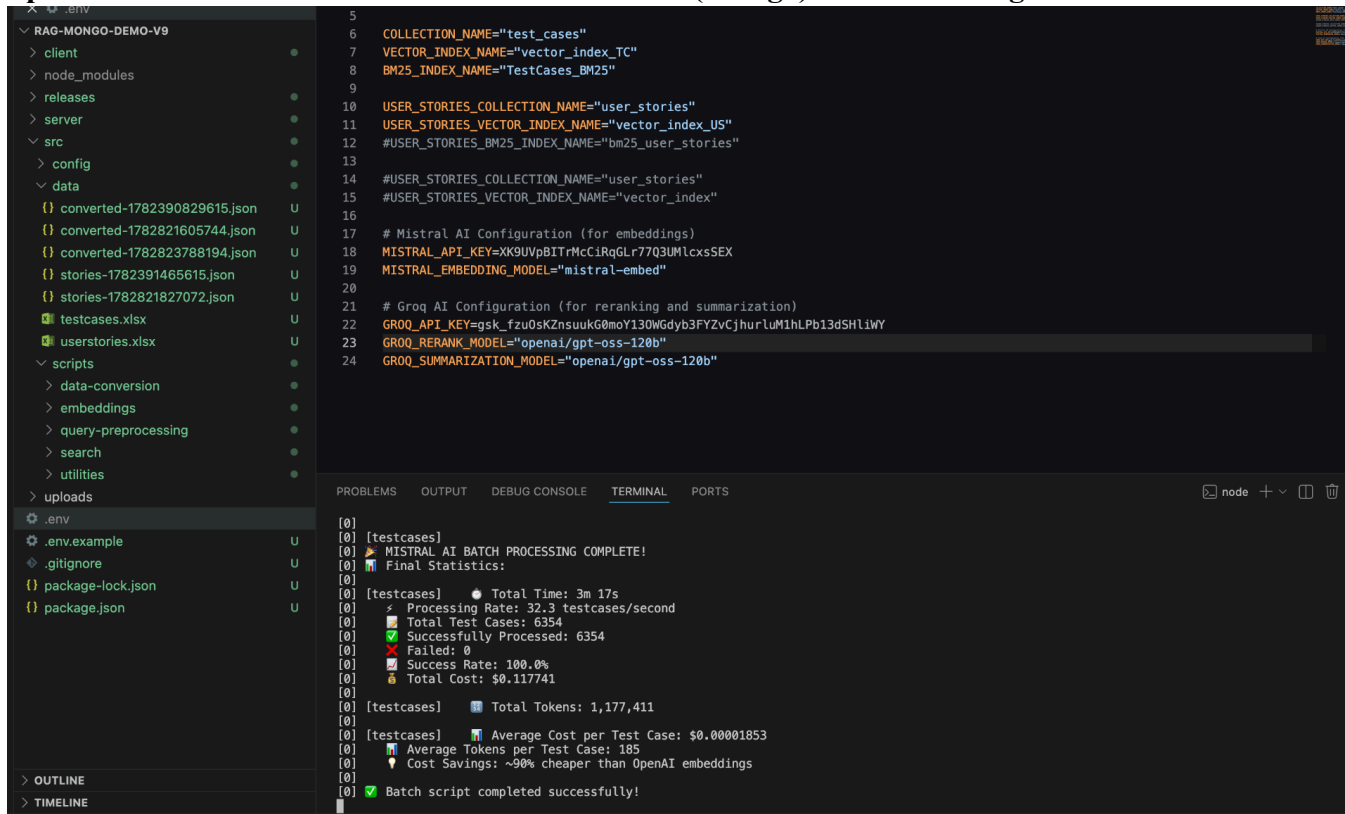
Key	Value	Description
file	userstories.xlsx	Excel file with user stories
dataType	userstories	
sheetName	stories	Sheet name inside the Excel file

Response Body (JSON):

```
{
  "success": true,
  "message": "User stories file converted successfully",
  "outputFile": "stories-1782821827072.json",
  "output": "📁 Converting 190 rows from sheet \"stories\"\\n✅ Converted 190 user stories from \"stories\" into /Users/ajayuchiha/Desktop/Gen AI/Phase 2/Week 4/Day 1/rag-mongo-demo-v9/src/data/stories-1782821827072.json\\n⚠️ Filtered out: 0 empty rows\\n"
}
```

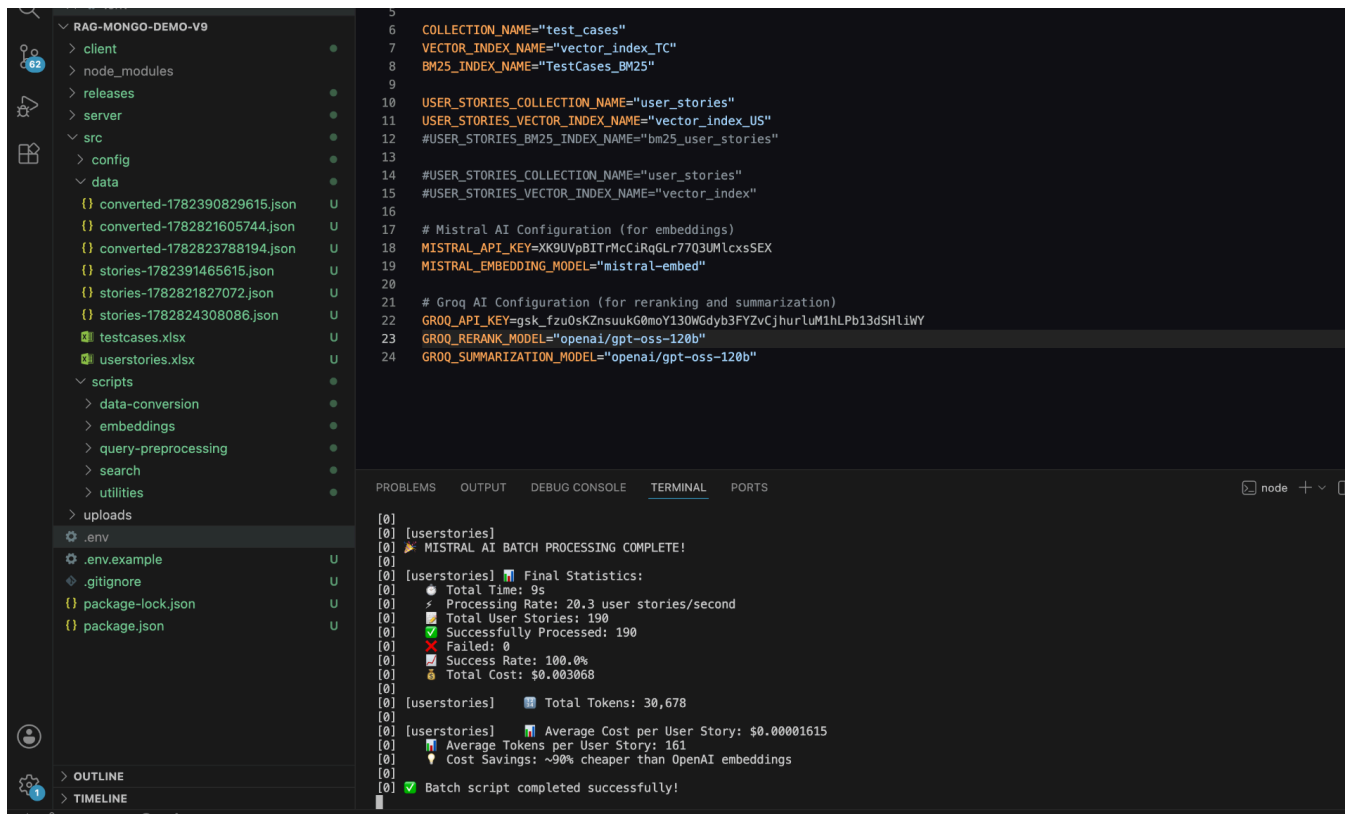
RAG INGESTION & RETRIEVAL USING API

2. Upload the converted JSON files to the Vector DB (Mongo) for Embedding:



```
5
6 COLLECTION_NAME="test_cases"
7 VECTOR_INDEX_NAME="vector_index_TC"
8 BM25_INDEX_NAME="TestCases_BM25"
9
10 USER_STORIES_COLLECTION_NAME="user_stories"
11 USER_STORIES_VECTOR_INDEX_NAME="vector_index_US"
12 #USER_STORIES_BM25_INDEX_NAME="bm25_user_stories"
13
14 #USER_STORIES_COLLECTION_NAME="user_stories"
15 #USER_STORIES_VECTOR_INDEX_NAME="vector_index"
16
17 # Mistral AI Configuration (for embeddings)
18 MISTRAL_API_KEY=sk-XK9UVp8ITrMcCIRqGLr77Q3UMLcxSEX
19 MISTRAL_EMBEDDING_MODEL="mistral-embed"
20
21 # Groq AI Configuration (for reranking and summarization)
22 GROQ_API_KEY=sk-fzu0sKZnsuukG0moY130WGdyb3FYZvCjhurluM1hLPb13dSHliWY
23 GROQ_RERANK_MODEL="openai/gpt-oss-120b"
24 GROQ_SUMMARIZATION_MODEL="openai/gpt-oss-120b"

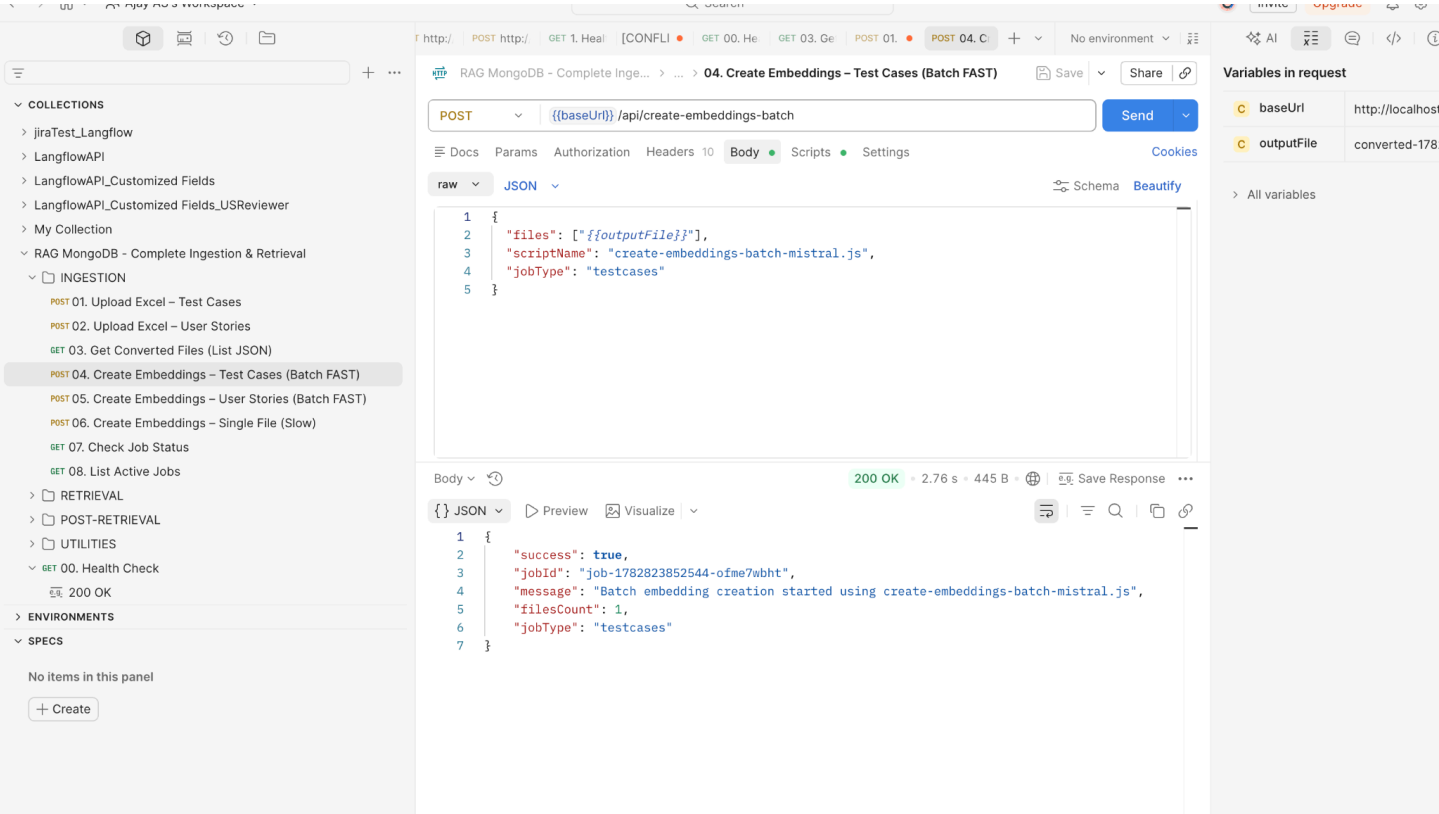
[0]
[0] [testcases]
[0] MISTRAL AI BATCH PROCESSING COMPLETE!
[0] Final Statistics:
[0]
[0] [testcases] ● Total Time: 3m 17s
[0] ✖ Processing Rate: 32.3 testcases/second
[0] 📊 Total Test Cases: 6354
[0] ✅ Successfully Processed: 6354
[0] ❌ Failed: 0
[0] 📈 Success Rate: 100.0%
[0] 💰 Total Cost: $0.117741
[0]
[0] [testcases] 📊 Total Tokens: 1,177,411
[0]
[0] [testcases] 📊 Average Cost per Test Case: $0.0001853
[0] 📊 Average Tokens per Test Case: 185
[0] ⬇ Cost Savings: ~90% cheaper than OpenAI embeddings
[0]
[0] ✅ Batch script completed successfully!
```



```
5
6 COLLECTION_NAME="test_cases"
7 VECTOR_INDEX_NAME="vector_index_TC"
8 BM25_INDEX_NAME="TestCases_BM25"
9
10 USER_STORIES_COLLECTION_NAME="user_stories"
11 USER_STORIES_VECTOR_INDEX_NAME="vector_index_US"
12 #USER_STORIES_BM25_INDEX_NAME="bm25_user_stories"
13
14 #USER_STORIES_COLLECTION_NAME="user_stories"
15 #USER_STORIES_VECTOR_INDEX_NAME="vector_index"
16
17 # Mistral AI Configuration (for embeddings)
18 MISTRAL_API_KEY=sk-XK9UVp8ITrMcCIRqGLr77Q3UMLcxSEX
19 MISTRAL_EMBEDDING_MODEL="mistral-embed"
20
21 # Groq AI Configuration (for reranking and summarization)
22 GROQ_API_KEY=sk-fzu0sKZnsuukG0moY130WGdyb3FYZvCjhurluM1hLPb13dSHliWY
23 GROQ_RERANK_MODEL="openai/gpt-oss-120b"
24 GROQ_SUMMARIZATION_MODEL="openai/gpt-oss-120b"

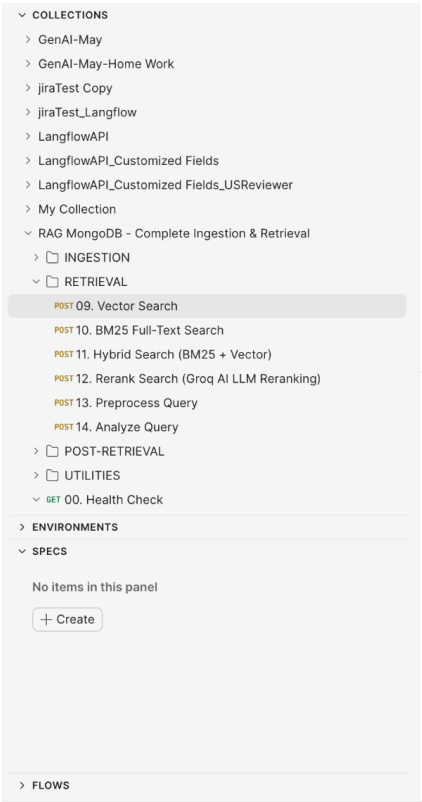
[0]
[0] [userstories]
[0] MISTRAL AI BATCH PROCESSING COMPLETE!
[0] Final Statistics:
[0]
[0] [userstories] ● Total Time: 9s
[0] ✖ Processing Rate: 20.3 user stories/second
[0] 📊 Total User Stories: 190
[0] ✅ Successfully Processed: 190
[0] ❌ Failed: 0
[0] 📈 Success Rate: 100.0%
[0] 💰 Total Cost: $0.003068
[0]
[0] [userstories] 📊 Total Tokens: 30,678
[0]
[0] [userstories] 📊 Average Cost per User Story: $0.0001615
[0] 📊 Average Tokens per User Story: 161
[0] ⬇ Cost Savings: ~90% cheaper than OpenAI embeddings
[0]
[0] ✅ Batch script completed successfully!
```

RAG INGESTION & RETRIEVAL USING API



RAG RETRIEVAL:

1. Vector Search:



RAG INGESTION & RETRIEVAL USING API

2. BM25 Search (Exact Match):

The screenshot displays the Langflow API interface. On the left, a sidebar shows a collection of documents, including 'GenAI-May', 'GenAI-May-Home Work', 'jiraTest Copy', 'jiraTest_Langflow', 'LangflowAPI', 'LangflowAPI_Customized Fields', 'LangflowAPI_Customized Fields_USReviewer', 'My Collection', 'RAG MongoDB - Complete Ingestion & Retrieval', 'INGESTION', 'RETRIEVAL', 'POST 09. Vector Search', 'POST 10. BM25 Full-Text Search', 'POST 11. Hybrid Search (BM25 + Vector)', 'POST 12. Rerank Search (Groq AI LLM Reranking)', 'POST 13. Preprocess Query', 'POST 14. Analyze Query', 'POST-RETRIEVAL', 'UTILITIES', 'GET 00. Health Check', 'ENVIRONMENTS', 'SPECS', and 'FLOWS'. The 'POST 10. BM25 Full-Text Search' endpoint is selected.

The main panel shows a POST request to the endpoint `{{baseUri}}/api/search/bm25`. The request body is a JSON object:

```
{
  "query": "{{searchQuery}}",
  "limit": {{searchLimit}},
  "filters": {
    "module": "",
    "priority": "",
    "risk": ""
  },
  "fields": [
    "id",
    "title",
    "description",
    "steps",
    "expectedResults"
  ]
}
```

The response is a 200 OK status with a response time of 2.85 s and a size of 8.06 KB. The response body is a JSON object:

```
{
  "results": [
    {
      "_id": "6a43bbb2ba07a5b36168110c",
      "module": "Admin",
      "id": "TC_2001",
      "title": "Functionality",
      "description": "To verify the user able to Create a Functionality to the Business module.",
      "steps": "1.Launch the application\r\n2.Login to the application with valid credentials.\r\n3.Click on External Service Tab and select Admin from the drop down.\r\n4.Click on Navigation Menu -> Masters ->Functionality.\r\n5.Click on the Functionality.\r\n6.Enter Functionality Name, Business Module and FunctionalityAlias Name.\r\n7.Click on \"SAVE\" button.",
      "expectedResults": "User can able to see the PopUp Message like Functionality Details added successfully and Record should Save.",
      "automationManual": "Manual",
      "priority": "P4",
      "risk": "Medium",
      "createdAt": "2026-06-30T12:50:58.446Z",
      "score": 14.268139839172363
    }
  ]
}
```

On the right, a sidebar shows the variables used in the request: `baseUrl` (http://localhost), `searchQuery` (login function), and `searchLimit` (10).

3. Hybrid (Vector 70 BM25 30):

The screenshot displays the Langflow API interface. On the left, a sidebar shows a collection of documents, including 'GenAI-May', 'GenAI-May-Home Work', 'jiraTest Copy', 'jiraTest_Langflow', 'LangflowAPI', 'LangflowAPI_Customized Fields', 'LangflowAPI_Customized Fields_USReviewer', 'My Collection', 'RAG MongoDB - Complete Ingestion & Retrieval', 'INGESTION', 'RETRIEVAL', 'POST 09. Vector Search', 'POST 10. BM25 Full-Text Search', 'POST 11. Hybrid Search (BM25 + Vector)', 'POST 12. Rerank Search (Groq AI LLM Reranking)', 'POST 13. Preprocess Query', 'POST 14. Analyze Query', 'POST-RETRIEVAL', 'UTILITIES', 'GET 00. Health Check', 'ENVIRONMENTS', 'SPECS', and 'FLOWS'. The 'POST 11. Hybrid Search (BM25 + Vector)' endpoint is selected.

The main panel shows a POST request to the endpoint `{{baseUri}}/api/search/hybrid`. The request body is a JSON object:

```
{
  "query": "{{searchQuery}}",
  "limit": {{searchLimit}},
  "bm25Weight": 0.3,
  "vectorWeight": 0.7,
  "filters": {
    "module": "",
    "priority": "",
    "risk": ""
  },
  "bm25Fields": [
    "id",
    "title",
    "description"
  ]
}
```

The response is a 200 OK status with a response time of 5.26 s and a size of 9.63 KB. The response body is a JSON object:

```
{
  "title": "Functionality",
  "description": "To verify the user able to Create a Functionality to the Business module.",
  "steps": "1.Launch the application\r\n2.Login to the application with valid credentials.\r\n3.Click on External Service Tab and select Admin from the drop down.\r\n4.Click on Navigation Menu -> Masters ->Functionality.\r\n5.Click on the Functionality.\r\n6.Enter Functionality Name, Business Module and FunctionalityAlias Name.\r\n7.Click on \"SAVE\" button.",
  "expectedResults": "User can able to see the PopUp Message like Functionality Details added successfully and Record should Save.",
  "automationManual": "Manual",
  "priority": "P4",
  "risk": "Medium",
  "createdAt": "2026-06-30T12:50:58.446Z",
  "bm25Score": 14.268139839172363,
  "bm25ScoreNormalized": 1,
  "vectorScore": 0.8829015493392944,
  "vectorScoreNormalized": 0.8829015493392944,
  "hybridScore": 0.9180310845375061,
  "foundIn": "both"
}
```

On the right, a sidebar shows the variables used in the request: `baseUrl` (http://localhost), `searchQuery` (login function), and `searchLimit` (10).

RAG INGESTION & RETRIEVAL USING API

4. Reranking the Search Context with the LLM:

The screenshot displays the Langflow interface for the Rerank Search (Groq AI LLM Reranking) step. The left sidebar shows a list of collections, including 'GenAI-May', 'GenAI-May-Home Work', 'jiraTest Copy', 'jiraTest_Langflow', 'LangflowAPI', 'LangflowAPI_Customized Fields', 'LangflowAPI_Customized Fields_USReviewer', 'My Collection', 'RAG MongoDB - Complete Ingestion & Retrieval', 'INGESTION', 'RETRIEVAL', 'POST 09. Vector Search', 'POST 10. BM25 Full-Text Search', 'POST 11. Hybrid Search (BM25 + Vector)', 'POST 12. Rerank Search (Groq AI LLM Reranking)', 'POST 13. Preprocess Query', 'POST 14. Analyze Query', 'POST-RETRIEVAL', 'UTILITIES', 'GET 00. Health Check', 'ENVIRONMENTS', and 'SPECS'. The main panel shows a POST request to the endpoint `{{baseUrl}}/api/search/rerank` with a JSON body. The response is a 200 OK status with a JSON body containing search results.

```
POST {{baseUrl}}/api/search/rerank
```

```
{
  "query": "{{searchQuery}}",
  "limit": {{searchLimit}},
  "rerankTopK": 50,
  "filters": {
    "module": "",
    "priority": "",
    "risk": ""
  }
}
```

```
{
  "_id": "6a43bc4bba07a5b36168244e",
  "module": "AHC",
  "id": "TC_2616",
  "title": "MasterSetup->Map Login Counter",
  "description": "To verify the user able to create Map Login Counter.",
  "steps": "\r\n1.Launch the application\r\n2.Login to the application with valid credentials.\r\n3.Click on Patient Service Tab and select AHC from drop down.\r\n4.Navigate to Functions -> MasterSetup\r\n5.Map Login Counter.\r\n6.Enter Login ID and select Counter.\r\n7.Click on \"ADD\" button.\r\n\r\n",
  "expectedResults": "User should able to create Map Login Counter.",
  "priority": "P1",
  "risk": "Critical",
  "bm25Score": 9.873544692993164,
  "rerankScore": 0.78,
  "originalRank": 9
}
```

5. Query PreProcessing:

The screenshot displays the Langflow interface for the Preprocess Query step. The left sidebar shows a list of collections, including 'GenAI-May', 'GenAI-May-Home Work', 'jiraTest Copy', 'jiraTest_Langflow', 'LangflowAPI', 'LangflowAPI_Customized Fields', 'LangflowAPI_Customized Fields_USReviewer', 'My Collection', 'RAG MongoDB - Complete Ingestion & Retrieval', 'INGESTION', 'RETRIEVAL', 'POST 09. Vector Search', 'POST 10. BM25 Full-Text Search', 'POST 11. Hybrid Search (BM25 + Vector)', 'POST 12. Rerank Search (Groq AI LLM Reranking)', 'POST 13. Preprocess Query', 'POST 14. Analyze Query', 'POST-RETRIEVAL', 'UTILITIES', 'GET 00. Health Check', 'ENVIRONMENTS', and 'SPECS'. The main panel shows a POST request to the endpoint `{{baseUrl}}/api/search/preprocess` with a JSON body. The response is a 200 OK status with a JSON body containing preprocessed query results.

```
POST {{baseUrl}}/api/search/preprocess
```

```
{
  "query": "{{searchQuery}}",
  "options": {
    "enableAbbreviations": true,
    "enableSynonyms": true,
    "maxSynonymVariations": 5,
    "smartExpansion": false,
    "preserveTestCaseIds": true
  }
}
```

```
{
  "original": "ER patient registration BP monitoring",
  "normalized": "er patient registration bp monitoring",
  "abbreviationExpanded": "emergency room patient registration blood pressure monitoring",
  "synonymExpanded": [
    "emergency room patient registration blood pressure monitoring",
    "urgent room patient registration blood pressure monitoring",
    "critical room patient registration blood pressure monitoring",
    "immediate room patient registration blood pressure monitoring",
    "emergency bed patient registration blood pressure monitoring"
  ],
  "metadata": {
    "testCaseIds": [],
    "abbreviationMappings": [
      {
        "abbreviation": "er",
        "expansion": "emergency room",
      }
    ]
  }
}
```

RAG INGESTION & RETRIEVAL USING API

RAG POST RETRIEVAL:

1. Summary (Detailed):

COLLECTIONS

LangflowAPI_Customized_Fields_USReviewer

My Collection

RAG MongoDB - Complete Ingestion & Retrieval

INGESTION

RETRIEVAL

POST 09. Vector Search

POST 10. BM25 Full-Text Search

POST 11. Hybrid Search (BM25 + Vector)

POST 12. Rerank Search (Groq AI LLM Reranking)

POST 13. Preprocess Query

POST 14. Analyze Query

POST-RETRIEVAL

POST 15. Summarize Results (Concise)

POST 16. Summarize Results (Detailed)

POST 17. Deduplicate Results

POST 18. Test AI Prompt (Groq)

UTILITIES

GET 00. Health Check

200 OK

RAG MongoDB - Data Ingestion

ENVIRONMENTS

SPECS

No items in this panel

+ Create

RAG MongoDB - Complete Ingestion & Retrieval > POST-RETRIEVAL > 16. Summarize Results (Detailed)

POST

{{baseUrl}}/api/search/summarize

Send

Docs Params Authorization Headers 10 Body Scripts Settings

raw JSON

Schema Beautify

```
1 {
2   "query": "{{searchQuery}}",
3   "summaryType": "detailed",
4   "results": [
5     {
6       "id": "TC_0001",
7       "module": "Login",
8       "title": "Verify valid user login",
9       "description": "Test that a valid user can log in successfully",
10      "steps": "1. Navigate to login page\n2. Enter valid credentials\n3. Click Login",
11      "expectedResults": "User is redirected to dashboard",
12      "priority": "High",
13      "risk": "High",
14      "automationManual": "Automation"
15    }
16  ]
17 }
```

Body

200 OK 2.77 s 5.15 KB

Save Response

JSON

Preview Visualize

```
1 {
2   "success": true,
3   "summary": "**Search Query:** \"Login functionality Test Cases\"\n**Search Results Returned:** 1 \n\n| # | ID | Title | Module | Description |
4     |-----|-----|
5     | 1 | TC_0001 | Verify valid user login | Login | Test that a valid user can log in successfully |
6     |-----|-----|
7     | 1. Quantitative Overview\nMetric | Value |-----|-----|
8     | Total results retrieved | **1** \n\nResults with exact phrase \"Login functionality\" in title | 0 \n\nResults with the word \"Login\" in title | **1** \n\nResults that explicitly mention \"Test Cases\" | Implicit (the record is a test case) \n\nResults that include a module tag | **1** (Login) \n\nResults that provide a description of expected behavior | **1** \n\nBecause only a single entry was returned, the statistical spread is limited, but the data still reveals useful patterns.
9     |-----|-----|
10    | 2. Qualitative Themes & Patterns\nTheme | Evidence from Result | Relevance to Query |
11    |-----|-----|
12    | **Login functionality** | Module = *Login*; Title mentions \"login\" | Directly matches the core
```

2. DeDuplication of Search Results:

COLLECTIONS

LangflowAPI_Customized_Fields_USReviewer

My Collection

RAG MongoDB - Complete Ingestion & Retrieval

INGESTION

RETRIEVAL

POST 09. Vector Search

POST 10. BM25 Full-Text Search

POST 11. Hybrid Search (BM25 + Vector)

POST 12. Rerank Search (Groq AI LLM Reranking)

POST 13. Preprocess Query

POST 14. Analyze Query

POST-RETRIEVAL

POST 15. Summarize Results (Concise)

POST 16. Summarize Results (Detailed)

POST 17. Deduplicate Results

POST 18. Test AI Prompt (Groq)

UTILITIES

GET 00. Health Check

200 OK

RAG MongoDB - Data Ingestion

ENVIRONMENTS

SPECS

No items in this panel

+ Create

RAG MongoDB - Complete Ingestion & Retrieval > POST-RETRIEVAL > 17. Deduplicate Results

POST

{{baseUrl}}/api/search/deduplicate

Send

Docs Params Authorization Headers 10 Body Scripts Settings

raw JSON

Schema Beautify

```
4 {
5   "id": "TC_0001",
6   "title": "Verify valid user login"
7 },
8 {
9   "id": "TC_0002",
10  "title": "Verify valid user login flow"
11 },
12 {
13   "id": "TC_0003",
14   "title": "Verify valid user login flow"
15 }
16 ]
17 }
```

Body

200 OK 3 ms 787 B

Save Response

JSON

Preview Visualize

```
21 {
22   "id": "TC_0002",
23   "title": "Verify valid user login flow"
24 },
25 ],
26 "duplicates": [
27   {
28     "id": "TC_0003",
29     "title": "Verify valid user login flow",
30     "duplicateOf": "TC_0002",
31     "similarity": "1.000"
32   }
33 ],
34 "stats": {
35   "originalCount": 3,
36   "deduplicatedCount": 2,
37   "duplicatesRemoved": 1,
```

RAG INGESTION & RETRIEVAL USING API

3. Testing with the AI Prompt:

The screenshot shows the Langflow API interface. On the left, the 'COLLECTIONS' panel lists various endpoints under 'RAG MongoDB - Complete Ingestion & Retrieval'. The 'POST-RETRIEVAL' section is expanded, showing endpoints like 'POST 09. Vector Search', 'POST 10. BM25 Full-Text Search', 'POST 11. Hybrid Search (BM25 + Vector)', 'POST 12. Rerank Search (Groq AI LLM Reranking)', 'POST 13. Preprocess Query', 'POST 14. Analyze Query', 'POST 15. Summarize Results (Concise)', 'POST 16. Summarize Results (Detailed)', 'POST 17. Deduplicate Results', and 'POST 18. Test AI Prompt (Groq)'. The 'POST 18. Test AI Prompt (Groq)' endpoint is selected.

The main panel shows the request details for the 'POST /api/test-prompt' endpoint. The request body is a JSON object:

```
{  "prompt": "Given the following test case titles, identify which ones cover edge cases:\n1. Verify valid user login\n2. Verify login with empty password\n3. Verify login with SQL injection\n4. Verify dashboard loads after login",  "temperature": 0.2,  "maxTokens": 2000}
```

The response is a 200 OK status with a JSON body:

```
{  "success": true,  "response": {    "raw": "**Edge-case test cases**\n\n**2. Verify login with empty password** \n *Why it's an edge case:* It tests a boundary condition (the minimal possible input for the password field) and checks how the system handles missing required data.\n\n**3. Verify login with SQL injection** \n *Why it's an edge case:* It probes an abnormal, potentially malicious input that lies far outside normal usage, testing the system's resilience to security attacks.\n\n**Not edge cases**\n\n**1. Verify valid user login** - a standard \"happy-path\" test. \n\n**4. Verify dashboard loads after login** - a functional verification of post-login behavior, not an extreme or boundary condition."  },    "model": "openai/gpt-oss-120b",    "provider": "groq-ai",    "tokens": {      "prompt": 115,      "completion": 300    }  }}
```

RAG API UTILITIES: Checking the Meta Data Filters:

The screenshot shows the Langflow API interface. On the left, the 'COLLECTIONS' panel lists various endpoints under 'RAG MongoDB - Complete Ingestion & Retrieval'. The 'UTILITIES' section is expanded, showing endpoints like 'GET 19. Get Metadata (Distinct Filter Values)', 'GET 20. Get Latest Test Case ID', 'GET 21. Get Environment Variables', and 'POST 22. Update Environment Variables'. The 'GET 19. Get Metadata (Distinct Filter Values)' endpoint is selected.

The main panel shows the request details for the 'GET /api/metadata/distinct' endpoint. The request body is empty. The response is a 200 OK status with a JSON body:

```
{  "success": true,  "metadata": {    "modules": [      "AHC",      "AT(Admission Module)",      "Admin",      "BB(Blood Bank)",      "Digital OP",      "Digital dietician",      "Doctors-Wards",      "Dynamic Forms",      "ES(Appointment)",      "Emergency",      "Human Resources (HR)",      "IP Billing",      "IP Digital",    ]  }}
```


RAG INGESTION & RETRIEVAL USING API

2. Fetching the Environmental Variables used:

The screenshot shows a REST client interface with a sidebar on the left listing collections and utilities. The main panel displays a GET request to `{{baseUri}} /api/env` with a status of **200 OK**. The response body is a JSON object containing the following environment variables:

```
{
  "MONGODB_URI": "mongodb+srv://ajayarumugaa_db_user:dIzViMvD4QPwRRC@clusteraiagentic01.h923scl.mongodb.net/?appName=ClusterAIAgentic01",
  "DB_NAME": "kb_TstGenAI",
  "COLLECTION_NAME": "test_cases",
  "VECTOR_INDEX_NAME": "vector_index_TC",
  "BM25_INDEX_NAME": "TestCases_BM25",
  "USER_STORIES_COLLECTION_NAME": "user_stories",
  "USER_STORIES_VECTOR_INDEX_NAME": "vector_index_US",
  "MISTRAL_API_KEY": "XK9UVpBItrMcCiRqGLr77Q3UMLcxSEX",
  "MISTRAL_EMBEDDING_MODEL": "mistral-embed",
  "GROQ_API_KEY": "gsk_fzu0sKZnsuukG0moY130WGdyb3FYZvCjhuirLuM1hLPb13dSHliwY",
  "GROQ_RERANK_MODEL": "openai/gpt-oss-120b",
  "GROQ_SUMMARIZATION_MODEL": "openai/gpt-oss-120b"
}
```

3. Getting the Test Cases Details from the Vector DB:

The screenshot shows a REST client interface with a sidebar on the left listing collections and utilities. The main panel displays a GET request to `{{baseUri}} /api/testcases/latest-id` with a status of **200 OK**. The response body is a JSON object containing the following test case details:

```
{
  "success": true,
  "latestId": 6354,
  "nextId": 6355,
  "nextTestCaseId": "TC_6355",
  "totalTestCases": 6354
}
```